

Отчёт по лабораторной работе №5

Имитационное моделирование

Ибрахим Мохсейн Алькамаль

Содержание

1	Цель работы	6
2	Выполнение лабораторной работы	7
2.1	Подготовка рабочего проекта	7
2.2	Проверка проектного окружения	7
2.3	Базовый эксперимент модели «Обедающие философы»	8
2.4	Преобразование базового эксперимента в литературный стиль	9
2.5	Выполнение базового эксперимента в Jupyter Notebook	9
2.6	Создание анимации модели «Обедающие философы»	10
2.7	Преобразование скрипта анимации в литературный стиль	10
2.8	Выполнение анимации в Jupyter Notebook	11
2.9	Формирование итогового графического отчёта	12
2.10	Преобразование итогового отчёта в литературный стиль	12
2.11	Сравнение классической сети и сети с арбитром	12
3	Моделирование задачи обедающих философов с помощью сетей Петри	14
3.1	Подключение рабочего окружения	14
3.2	Параметры моделирования	15
4	Классическая сеть Петри	16
4.1	Построение сети	16
4.2	Стохастическое моделирование	16
4.3	Сохранение результатов	17
4.4	Проверка взаимной блокировки	17
4.5	Визуализация динамики	18
5	Сеть Петри с арбитром	19
5.1	Построение сети с арбитром	19
5.2	Стохастическое моделирование	20
5.3	Сохранение результатов	22
5.4	Проверка взаимной блокировки	22
5.5	Визуализация	22
6	Вывод	23

7	Анимация сети Петри для задачи обедающих философов	24
7.1	Подключение рабочего окружения	24
7.2	Параметры моделирования	25
7.3	Построение классической сети Петри	25
7.4	Настройка воспроизводимости	26
7.5	Стохастическая симуляция	26
8	Создание анимации	27
8.1	Сохранение анимации	28
9	Вывод	30
10	Сравнительный анализ классической сети и сети с арбитром	31
10.1	Подключение рабочего окружения	31
10.2	Загрузка результатов моделирования	32
10.3	Количество философов	34
10.4	Выбор позиций состояния «Ест»	34
11	Классическая сеть Петри	35
12	Сеть Петри с арбитром	37
12.1	Объединение графиков	38
12.2	Сохранение итогового графика	39
13	Вывод	41
14	Выводы	42

Список иллюстраций

2.1	Создание рабочего проекта и установка зависимостей Julia	7
2.2	Проверка установленных пакетов проектного окружения Julia	8
2.3	Выполнение моделирования классической сети Петри и сети с арбитром	8
2.4	Генерация Julia-скрипта, документа Quarto и Jupyter Notebook для модели «Обедающие философы»	9
2.5	Результаты стохастического моделирования сети Петри с арбитром в Jupyter Notebook	10
2.6	Создание и сохранение анимации моделирования сети Петри	10
2.7	Генерация файлов литературной версии анимации модели «Обедающие философы»	11
2.8	Создание GIF-анимации изменения маркировки сети Петри в Jupyter Notebook	11
2.9	Формирование итогового графического отчёта по моделированию «Обедающих философов»	12
2.10	Генерация файлов литературной версии итогового анализа модели .	12
2.11	Сравнение результатов моделирования классической сети Петри и сети с арбитром	13

Список таблиц

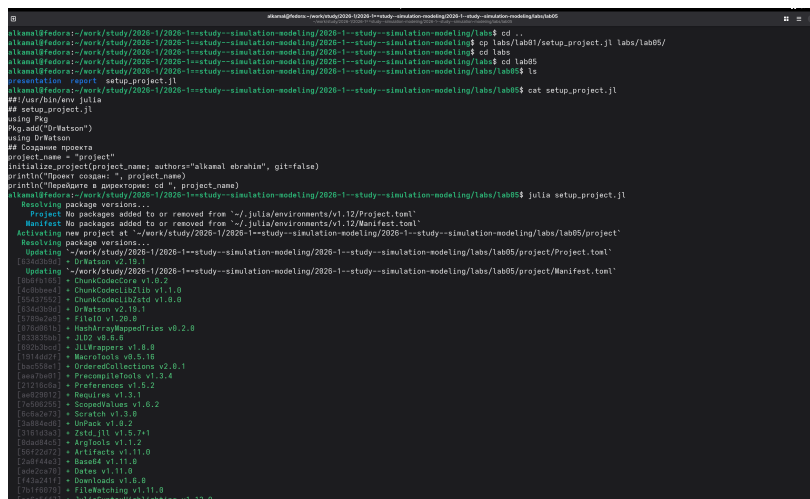
1 Цель работы

Изучить аппарат сетей Петри на примере задачи «Обедающие философы». Реализовать классическую сеть и сеть с арбитром, выполнить их моделирование, исследовать возникновение взаимной блокировки и визуализировать результаты.

2 Выполнение лабораторной работы

2.1 Подготовка рабочего проекта

Для выполнения лабораторной работы был создан проект lab05/project с помощью скрипта `setup_project.jl`. После инициализации проектного окружения были установлены необходимые зависимости Julia (рис. 2.1).



```
alkanal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/lab05$ cd ..
alkanal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling$ cp lab07/setup_project.jl lab05/
alkanal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling$ cd lab05
alkanal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/lab05$ cd lab05
alkanal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/lab05$ ls
presentation  report  setup_project.jl
alkanal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/lab05$ cat setup_project.jl
#!/usr/bin/env julia
## setup_project.jl
using Pkg
Pkg.add("Watson")
using DrWatson
## Создание проекта
project_name = "project"
initialize_project(project_name; authors="alkanal sbrahin", git=false)
println("Project name: ", project_name)
println("Project path: ", project_path)
alkanal@fedora:~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/lab05$ julia setup_project.jl
Resolving package versions...
Project: No packages added to or removed from "~/.julia/environments/v1.12/Project.toml"
Manifest: No packages added to or removed from "~/.julia/environments/v1.12/Manifest.toml"
Installing new project at "~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/lab05/project"
Resolving package versions...
Updating "~/work/study/2026-1/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/lab05/project/Manifest.toml"
[564030d] + Base v1.12.0
[4c8b9ac] + ChunkCode v1.0.2
[564030d] + FileIO v1.10.0
[564030d] + JLD2 v0.8.0
[564030d] + JLD v0.10.0
[564030d] + MacroTools v0.8.0
[564030d] + OrderedCollections v2.0.1
[564030d] + Preferences v1.5.2
[564030d] + Requires v1.3.1
[564030d] + Sequences v1.6.2
[564030d] + Scratch v1.3.0
[564030d] + UnPack v1.8.2
[564030d] + Zstd.jl v1.5.7+1
[564030d] + Artifacts v1.1.0
[564030d] + Dates v1.11.0
[564030d] + Downloads v1.8.0
[564030d] + FileWatching v1.11.0
[564030d] + JuliaSyntaxHighlighting v1.12.0
```

Рисунок 2.1: Создание рабочего проекта и установка зависимостей Julia

2.2 Проверка проектного окружения

Проектное окружение было запущено командой `julia --project=.`. С помощью `Pkg.status()` проверено наличие установленных пакетов, необходимых для выполнения моделирования и обработки результатов (рис. 2.2).

```
[1460000] • MPI v1.11.0
[1460020] • Serialization v1.11.0
[1460040] • SharedArrays v1.11.0
[1460060] • Sockets v1.11.0
[1460080] • SparseArrays v1.12.0
[1460100] • SuiteSparse
[1460120] • Test v1.11.0
[1460140] • CompilerSupportLibraries.jll v1.3.0+1
[1460160] • OpenBLAS.jll v0.3.29+0
[1460180] • OpenLibm.jll v0.8.7+0
[1460200] • Pkg2.jll v0.44.0+1
[1460220] • SuiteSparse.jll v7.8.3+2
[1460240] • liblapacke.jll v5.15.0+0
Info: Packages marked with * have new versions available but compatibility constraints restrict them from upgrading. To see why use 'status --outdated -m'

Две проверки: using DrWatson, DifferentialEquations, Plots
stknal@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ julia --project-
┌──────────┴──────────┐ Documentation: https://docs.julialang.org
┌──────────┴──────────┐ Type "???" for help, "???" for Pkg help.
┌──────────┴──────────┐ Version 1.12.1 (2025-10-17)
└──────────┴──────────┘ Fedora 44 build
julia> using Pkg
julia> Pkg.status()
status ~="/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project#Project.toml"
[1460260] Agents v7.8.3
[1460280] BenchmarkTools v1.8.0
[1460300] BlackboxOptim v0.6.12
[1460320] CSV v0.10.17
[1460340] DataFrames v1.8.2
[1460360] DifferentialEquations v8.1.1
[1460380] Distributions v0.25.131
[1460400] DrWatson v2.15.1
[1460420] Julia v1.12.1
[1460440] NIMBLE v2.12.0
[1460460] Literate v2.21.0
[1460480] OrdinaryDiffEq v7.8.1
[1460500] Plots v1.41.2
[1460520] Quarto v1.8.0
[1460540] StatsBase v0.34.13
[1460560] StatsPlots v0.15.8
[1460580] Dates v1.11.0
julia>
```

Рисунок 2.2: Проверка установленных пакетов проектного окружения Julia

2.3 Базовый эксперимент модели «Обедающие философы»

После подготовки модуля DiningPhilosophers.jl и скрипта dining_philosophers.jl выполнено стохастическое моделирование классической сети и сети с арбитром. Для классической сети обнаружено состояние взаимной блокировки (Deadlock обнаружен: true), а для сети с арбитром взаимная блокировка отсутствует (Deadlock обнаружен: false). Результаты сохранены в CSV-файлах и графиках (рис. 2.3).

```
stknal@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ nano src/DiningPhilosophers.jl
stknal@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ nano scripts/dining_philosophers.jl
stknal@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ julia --project- scripts/dining_philosophers.jl
== Классическая сеть (без арбтра) ==
Deadlock обнаружен: true

== Сеть с арбитром ==
Could not create decoration from factory! Running with no decorations.
Deadlock обнаружен: false
stknal@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ ls plots/
arbitr_simulation.png classic_simulation.png
stknal@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$ ls data/
dining_arbitr.csv dining_classic.csv exp_pro exp_rae sims
stknal@fedora:~/work/study/2026-1--study--simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project$
```

Рисунок 2.3: Выполнение моделирования классической сети Петри и сети с арбитром

Для документирования эксперимента подготовлен литературный скрипт `dining_philosophers_literary.jl`. С помощью `tangle.jl` из него автоматически сформированы чистый Julia-скрипт, документ Quarto и Jupyter Notebook (рис. 2.4).

```

phosphors.library.jl
[osmium w2 scripts/dining.phosphors.library.jl]
[info] generating plain script from ~"/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining.phosphors.library.jl"
[info] writing result to ~"/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining.phosphors.library.dining.phosphors.library.jl"
[warn] export: /home/aisaka/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining.phosphors.library.dining.phosphors.library.jl
[info] generating markdown page from ~"/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining.phosphors.library.dining.phosphors.library.jl"
[info] writing result to ~"/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining.phosphors.library.dining.phosphors.library.jl"
[warn] Quarto: /home/aisaka/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/markdown/dining.phosphors.library.dining.phosphors.library.jl
[info] generating notebook from ~"/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/scripts/dining.phosphors.library.dining.phosphors.library.jl"
[info] writing result to ~"/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/notebooks/dining.phosphors.library.dining.phosphors.library.ipynb"
[warn] Notebook: /home/aisaka/work/study/2026-1/2026-1-study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/notebooks/dining.phosphors.library.dining.phosphors.library.ipynb

[osmium Re Admin console]

```

2.5 Выполнение базового эксперимента в Jupyter Notebook

Сгенерированный `dining_philosophers_literary.ipynb` выполнен в Jupyter Notebook. В ноутбуке проведено стохастическое моделирование сети с арбитром и получена таблица изменения состояний Think, Hungry и Eat для пяти философов во времени. Результаты сохраняются в файл `data/dining_arbiter.csv` (рис. 2.5).

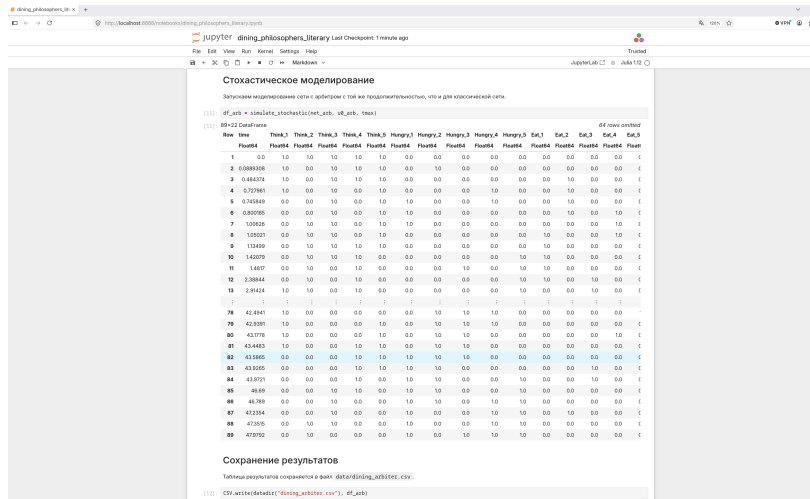


Рисунок 2.5: Результаты стохастического моделирования сети Петри с арбитром в Jupyter Notebook

2.6 Создание анимации модели «Обедающие философы»

Для визуализации изменения состояний сети Петри выполнен скрипт `dining_philosophers_animation.jl`. В результате создана GIF-анимация `philosophers_simulation.gif` и сохранена в каталоге `plots` (рис. 2.6).

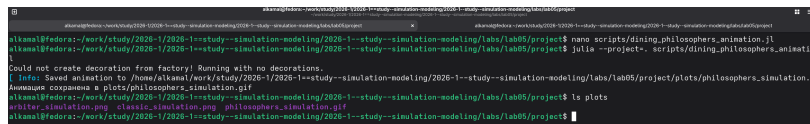


Рисунок 2.6: Создание и сохранение анимации моделирования сети Петри

2.7 Преобразование скрипта анимации в литературный стиль

Для документирования процесса визуализации подготовлен файл `dining_philosophers_animation_literary.jl`. С помощью `tangle.jl` автоматически

сформированы чистый Julia-скрипт, документ Quarto и Jupyter Notebook (рис. 2.7).

[illegible]

Рисунок 2.7: Генерация файлов литературной версии анимации модели «Обедающие философы»

2.8 Выполнение анимации в Jupyter Notebook

Сгенерированный `dining_philosophers_animation_literary.ipynb` выполнен в Jupyter Notebook. Последовательность кадров сохранена в формате GIF с частотой два кадра в секунду в файле `plots/philosophers_simulation.gif`. Кадры отображают изменение маркировки позиций сети Петри во времени (рис. 2.8).

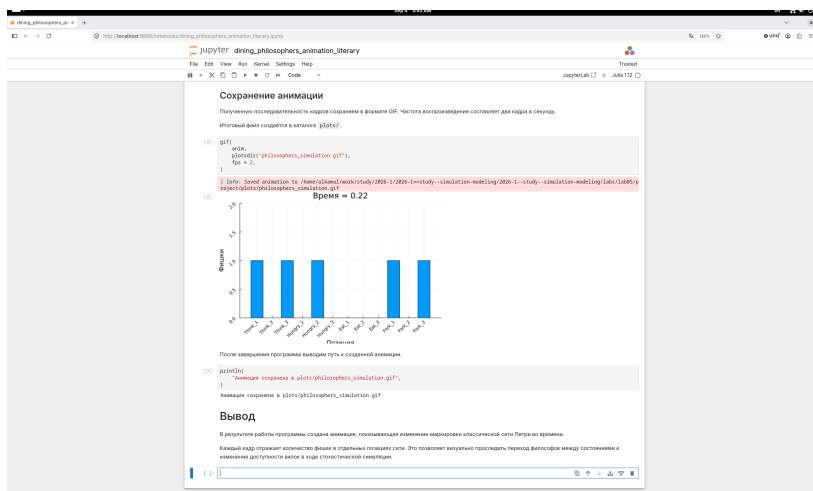


Рисунок 2.8: Создание GIF-анимации изменения маркировки сети Петри в Jupyter Notebook

2.9 Формирование итогового графического отчёта

Для сравнения результатов классической сети и сети с арбитром выполнен скрипт `dining_philosophers_report.jl`. В результате сформирован итоговый график `final_report.png`, сохранённый в каталоге `plots` (рис. 2.9).

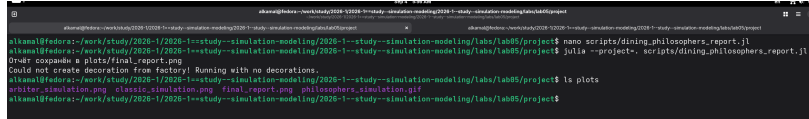


Рисунок 2.9: Формирование итогового графического отчёта по моделированию «Обедающих философов»

2.10 Преобразование итогового отчёта в литературный стиль

Для документирования итогового анализа подготовлен файл `dining_philosophers_report_literary.jl`. С помощью `tangle.jl` автоматически сформированы чистый Julia-скрипт, документ Quarto и Jupyter Notebook (рис. 2.10).

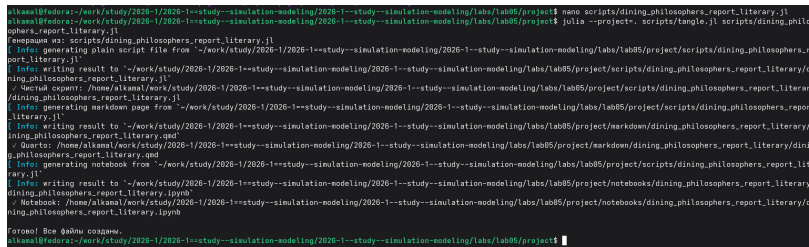


Рисунок 2.10: Генерация файлов литературной версии итогового анализа модели

2.11 Сравнение классической сети и сети с арбитром

Сгенерированный `dining_philosophers_report_literary.ipynb` выполнен в Jupyter Notebook. Для сравнения объединены графики классической сети и сети с

арбитром в одну вертикальную композицию, отображающую состояния философов во времени (рис. 2.11).

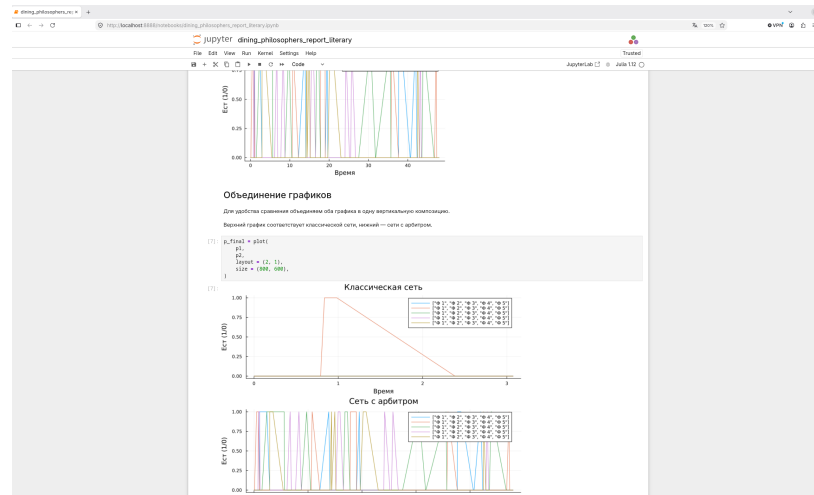


Рисунок 2.11: Сравнение результатов моделирования классической сети Петри и сети с арбитром

3 Моделирование задачи обедающих философов с помощью сетей Петри

В данной программе проводится сравнительное моделирование двух вариантов сети Петри для задачи обедающих философов:

1. классической сети без дополнительного механизма синхронизации;
2. модифицированной сети с арбитром.

Для каждого варианта выполняется стохастическая симуляция, результаты сохраняются в CSV-файл, проверяется наличие взаимной блокировки (deadlock), а также строится график изменения маркировки сети во времени.

3.1 Подключение рабочего окружения

Активируем проект DrWatson и подключаем модуль с реализацией сетей Петри для задачи обедающих философов.

```
using DrWatson  
  
@quickactivate "project"
```

```
ENV["GKSwstype"] = "100"

include(scrdir("DiningPhilosophers.jl"))

using .DiningPhilosophers

using DataFrames, CSV, Plots
```

3.2 Параметры моделирования

В классической постановке рассматриваются пять философов. Продолжительность стохастического моделирования зададим равной 50 единицам модельного времени.

```
N = 5
tmax = 50.0
```

50.0

4 Классическая сеть Петри

Сначала исследуем классический вариант задачи без арбитра. Такая схема допускает возникновение взаимной блокировки, когда каждый философ удерживает одну вилку и ожидает освобождения второй.

```
println("=== Классическая сеть (без арбитра) ===")
```

```
=== Классическая сеть (без арбитра) ===
```

4.1 Построение сети

Функция `build_classical_network` формирует сеть Петри для N философов и возвращает объект сети, начальную маркировку и имена позиций.

```
net_classic, u0_classic, _ = build_classical_network(N)
```

```
(Main.Notebook.DiningPhilosophers.PetriNet(20, 15, [-1 0 ... 0 0; 0 -1 ... 0 0; ... ; 0 0 ... 1 0; 0
```

4.2 Стохастическое моделирование

Выполняем стохастическую симуляцию классической сети до момента $t_{\max}$. Полученные маркировки сохраняются в таблице `DataFrame`.


```
df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
```

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0
2	0.621029	1.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0
3	0.729781	1.0	0.0	1.0	1.0	0.0	0.0	1.0	0.0
4	0.857894	1.0	0.0	1.0	1.0	0.0	0.0	1.0	0.0
5	1.65701	1.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0
6	1.99512	1.0	0.0	0.0	0.0	0.0	0.0	1.0	1.0
7	2.32348	1.0	0.0	0.0	0.0	1.0	0.0	1.0	1.0
8	2.45974	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0
9	3.10233	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0
10	3.22915	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0
11	3.55084	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0
12	4.7644	0.0	0.0	0.0	0.0	0.0	1.0	1.0	1.0

4.3 Сохранение результатов

Результаты классического варианта сохраняются в CSV-файл `data/dining_classic.csv`.

```
CSV.write(datadir("dining_classic.csv"), df_classic)
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab05/project/data/dining_classic.csv"
```

4.4 Проверка взаимной блокировки

С помощью функции `detect_deadlock` проверяем, достигла ли сеть состояния, в котором дальнейшее срабатывание переходов невозможно.

```
dead = detect_deadlock(df_classic, net_classic)

println("Deadlock обнаружен: $dead")
```

Deadlock обнаружен: true

4.5 Визуализация динамики

Строим график изменения маркировки классической сети во времени и сохраняем его в каталоге plots.

```
plot_classic = plot_marking_evolution(df_classic, N)

savefig(plotsdir("classic_simulation.png"))
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab05/project/plots/classic_simulation.png"
```

5 Сеть Петри с арбитром

Далее исследуем модифицированный вариант сети. В него вводится дополнительный арбитр, ограничивающий число философов, которые могут одновременно приступить к захвату ресурсов.

Это позволяет предотвратить ситуацию, при которой все философы одновременно удерживают по одной вилке.

```
println("\n=== Сеть с арбитром ===")
```

```
=== Сеть с арбитром ===
```

5.1 Построение сети с арбитром

Функция `build_arbiter_network` создаёт сеть Петри с дополнительной позицией, представляющей ресурс арбитра.

```
net_arb, u0_arb, _ = build_arbiter_network(N)
```

```
(Main.Notebook.DiningPhilosophers.PetriNet(21, 15, [-1 0 ... 0 0; 0 -1 ... 0 0; ... ; 0 0 ... 1 1; -  
1 -1 ... 1 1], [:Think_1, :Think_2, :Think_3, :Think_4, :Think_5, :Hungry_1, :Hungry_2, :Hungry_3, :Hungry_4, :Hungry_5, :Arbiter]))
```

5.2 Стохастическое моделирование

Запускаем моделирование сети с арбитром с той же продолжительностью, что и для классической сети.

```
df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
```

	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0
2	0.0384356	1.0	1.0	1.0	0.0	1.0	0.0	0.0	0.0
3	0.159993	0.0	1.0	1.0	0.0	1.0	1.0	0.0	0.0
4	0.227672	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0
5	0.755359	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0
6	1.70549	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0
7	4.33407	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0
8	4.65019	0.0	0.0	0.0	1.0	1.0	1.0	1.0	0.0
9	5.13744	0.0	0.0	1.0	1.0	1.0	1.0	1.0	0.0
10	5.32739	0.0	0.0	1.0	1.0	1.0	1.0	0.0	0.0
11	5.35647	0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0
12	6.49203	0.0	1.0	1.0	1.0	0.0	1.0	0.0	0.0
13	6.52223	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0
14	6.57308	0.0	0.0	1.0	0.0	0.0	1.0	1.0	0.0
15	6.96359	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0
16	7.31108	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0
17	7.35408	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0
18	8.18305	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
19	8.71869	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
20	8.97435	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
21	9.24706	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0
22	9.33459	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
23	10.2131	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
24	10.2998	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0
...	...	...	...	...	...	...	...	...	...

5.3 Сохранение результатов

Таблица результатов сохраняется в файл `data/dining_arbiter.csv`.

```
CSV.write(datadir("dining_arbiter.csv"), df_arb)
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab05/project/data/dining_arbiter.csv"
```

5.4 Проверка взаимной блокировки

Проверяем наличие deadlock для сети с арбитром.

```
dead_arb = detect_deadlock(df_arb, net_arb)  
  
println("Deadlock обнаружен: $dead_arb")
```

```
Deadlock обнаружен: false
```

5.5 Визуализация

Строим график изменения маркировки сети с арбитром и сохраняем его отдельно для последующего сравнения с классическим вариантом.

```
plot_arb = plot_marking_evolution(df_arb, N)  
  
savefig(plotsdir("arbiter_simulation.png"))
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--  
study--simulation-modeling/labs/lab05/project/plots/arbiter_simulation.png"
```

6 Вывод

В результате выполнения программы получены два стохастических эксперимента для одной и той же задачи.

Классическая сеть позволяет исследовать возможность возникновения взаимной блокировки при конкурентном доступе философов к вилкам.

Вариант с арбитром вводит дополнительное ограничение на захват ресурсов и используется как механизм предотвращения deadlock.

Полученные CSV-файлы и графики позволяют сравнить динамику маркировок двух вариантов сети Петри и оценить эффективность введения арбитра.

7 Анимация сети Петри для задачи обедающих философов

В данном примере создаётся анимация изменения маркировки классической сети Петри для задачи обедающих философов.

Для уменьшения числа отображаемых позиций используется сеть из трёх философов. Стохастическая симуляция выполняется в течение 30 единиц модельного времени.

На каждом кадре анимации отображается текущее количество фишек в каждой позиции сети Петри.

7.1 Подключение рабочего окружения

Активируем окружение проекта DrWatson и подключаем модуль DiningPhilosophers, содержащий функции построения сети и стохастического моделирования.

```
using DrWatson

@quickactivate "project"

include(sourcedir("DiningPhilosophers.jl"))

using .DiningPhilosophers
```


Для визуализации используется пакет Plots, а пакет Random необходим для задания фиксированного зерна генератора случайных чисел.

```
using Plots, Random
```

7.2 Параметры моделирования

В данной визуализации рассматриваются три философа. Длительность моделирования составляет 30 единиц времени.

```
N = 3  
tmax = 30.0
```

30.0

7.3 Построение классической сети Петри

С помощью функции `build_classical_network` создаём классическую сеть Петри без арбитра.

Функция возвращает:

- `net` — структуру сети Петри;
- `u0` — начальную маркировку;
- `names` — имена позиций сети.

```
net, u0, names = DiningPhilosophers.build_classical_network(N)
```

```
(Main.Notebook.DiningPhilosophers.PetriNet(12, 9, [-1 0 ... 0 0; 0 -1 ... 1 0; ... ; 0 -  
1 ... 1 0; 0 0 ... 1 1], [:Think_1, :Think_2, :Think_3, :Hungry_1, :Hungry_2, :Hungry_3, :Eat_1,
```

7.4 Настройка воспроизводимости

Поскольку стохастическая симуляция использует случайные числа, задаём фиксированное зерно генератора. Это позволяет получать одинаковую траекторию при повторном запуске программы.

```
Random.seed!(123)
```

```
TaskLocalRNG()
```

7.5 Стохастическая симуляция

Выполняем моделирование классической сети Петри до момента времени $t_{\max}$.

Результат сохраняется в таблицу `df`, где каждая строка соответствует одной маркировке сети в определённый момент времени.

```
df = DiningPhilosophers.simulate_stochastic(net, u0, tmax)
```

	time	Think_1	Think_2	Think_3	Hungry_1	Hungry_2	Hungry_3	Eat_1	Eat_2
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64
1	0.0	1.0	1.0	1.0	0.0	0.0	0.0	0.0	0.0
2	0.217198	1.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0
3	0.255714	0.0	0.0	1.0	1.0	1.0	0.0	0.0	0.0
4	0.577262	0.0	0.0	0.0	1.0	1.0	1.0	0.0	0.0

8 Создание анимации

Для каждой строки таблицы результатов создаётся отдельный кадр.

Значения всех колонок, кроме `time`, представляют количество фишек в соответствующих позициях сети.

```
anim = @animate for row in eachrow(df)

    u = [
        row[col]
        for col in propertynames(row)
        if col != :time
    ]

    bar(
        1:length(u),
        u,
        legend = false,
        ylims = (0, maximum(u0) + 1),
        xlabel = "Позиция",
        ylabel = "Фишки",
        title = "Время = $(round(row.time, digits = 2))",
```

```

)

xticks!(
    1:length(u),
    string.(names),
    rotation = 45,
)

end

```

```
Animation("/tmp/jl_nczjbY", ["000001.png", "000002.png", "000003.png", "000004.png"])
```

8.1 Сохранение анимации

Полученную последовательность кадров сохраняем в формате GIF. Частота воспроизведения составляет два кадра в секунду.

Итоговый файл создаётся в каталоге `plots/`.

```

gif(
    anim,
    plotsdir("philosophers_simulation.gif"),
    fps = 2,
)

```

```
[ Info: Saved animation to /home/alkamal/work/study/2026-1/2026-1==study--
simulation-modeling/2026-1--study--simulation-modeling/labs/lab05/project/plots/philosophers
Plots.AnimatedGif("/home/alkamal/work/study/2026-1/2026-1==study--simulation-
modeling/2026-1--study--simulation-modeling/labs/lab05/project/plots/philosophers_simulation

```

После завершения программы выводим путь к созданной анимации.

```
println(  
    "Анимация сохранена в plots/philosophers_simulation.gif",  
)
```

Анимация сохранена в plots/philosophers_simulation.gif

9 Вывод

В результате работы программы создана анимация, показывающая изменение маркировки классической сети Петри во времени.

Каждый кадр отражает количество фишек в отдельных позициях сети. Это позволяет визуально проследить переход философов между состояниями и изменение доступности вилок в ходе стохастической симуляции.

10 Сравнительный анализ

классической сети и сети с

арбитром

В данном скрипте выполняется сравнительный анализ результатов моделирования задачи обедающих философов для двух вариантов сети Петри:

1. классической сети без арбитра;
2. сети с арбитром.

Для анализа используются ранее сохранённые CSV-файлы. Основное внимание уделяется состоянию `Eat_i`, которое показывает, находится ли соответствующий философ в состоянии приёма пищи.

10.1 Подключение рабочего окружения

Активируем окружение проекта DrWatson и подключаем библиотеки для работы с таблицами, CSV-файлами и построения графиков.

```
using DrWatson
@quickactivate "project"

using DataFrames, CSV, Plots
```

10.2 Загрузка результатов моделирования

Загружаем результаты классической сети Петри и сети с арбитром.

Файлы были получены на предыдущем этапе моделирования и содержат временную динамику маркировок всех позиций сети.

```
df_classic = CSV.read(  
    datadir("dining_classic.csv"),  
    DataFrame,  
)  
  
df_arbiter = CSV.read(  
    datadir("dining_arbiter.csv"),  
    DataFrame,  
)
```


	time	Think_1	Think_2	Think_3	Think_4	Think_5	Hungry_1	Hungry_2	Hungry_3
	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64	Float64
1	0.0	1.0	1.0	1.0	1.0	1.0	0.0	0.0	0.0
2	0.0384356	1.0	1.0	1.0	0.0	1.0	0.0	0.0	0.0
3	0.159993	0.0	1.0	1.0	0.0	1.0	1.0	0.0	0.0
4	0.227672	0.0	1.0	0.0	0.0	1.0	1.0	0.0	1.0
5	0.755359	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0
6	1.70549	0.0	0.0	0.0	0.0	1.0	1.0	1.0	1.0
7	4.33407	0.0	0.0	0.0	1.0	1.0	1.0	1.0	1.0
8	4.65019	0.0	0.0	0.0	1.0	1.0	1.0	1.0	0.0
9	5.13744	0.0	0.0	1.0	1.0	1.0	1.0	1.0	0.0
10	5.32739	0.0	0.0	1.0	1.0	1.0	1.0	0.0	0.0
11	5.35647	0.0	0.0	1.0	1.0	0.0	1.0	0.0	0.0
12	6.49203	0.0	1.0	1.0	1.0	0.0	1.0	0.0	0.0
13	6.52223	0.0	0.0	1.0	1.0	0.0	1.0	1.0	0.0
14	6.57308	0.0	0.0	1.0	0.0	0.0	1.0	1.0	0.0
15	6.96359	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0
16	7.31108	0.0	1.0	1.0	0.0	0.0	1.0	0.0	0.0
17	7.35408	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0
18	8.18305	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
19	8.71869	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
20	8.97435	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
21	9.24706	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0
22	9.33459	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
23	10.2131	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0
24	10.2998	1.0	1.0	0.0	0.0	1.0	0.0	0.0	1.0
...	...	...	...	...	...	...	...	...	...

10.3 Количество философов

В рассматриваемой модели используется пять философов.

```
N = 5
```

5

10.4 Выбор позиций состояния «Ест»

Для каждого философа в сети существует позиция Eat_i, которая принимает значение 1, когда философ ест, и 0 — в остальных состояниях.

Формируем список соответствующих столбцов таблицы:

Eat_1, Eat_2, ..., Eat_5.

```
eat_cols = [  
    Symbol("Eat_$i")  
    for i = 1:N  
]
```

5-element Vector{Symbol}:

:Eat_1

:Eat_2

:Eat_3

:Eat_4

:Eat_5

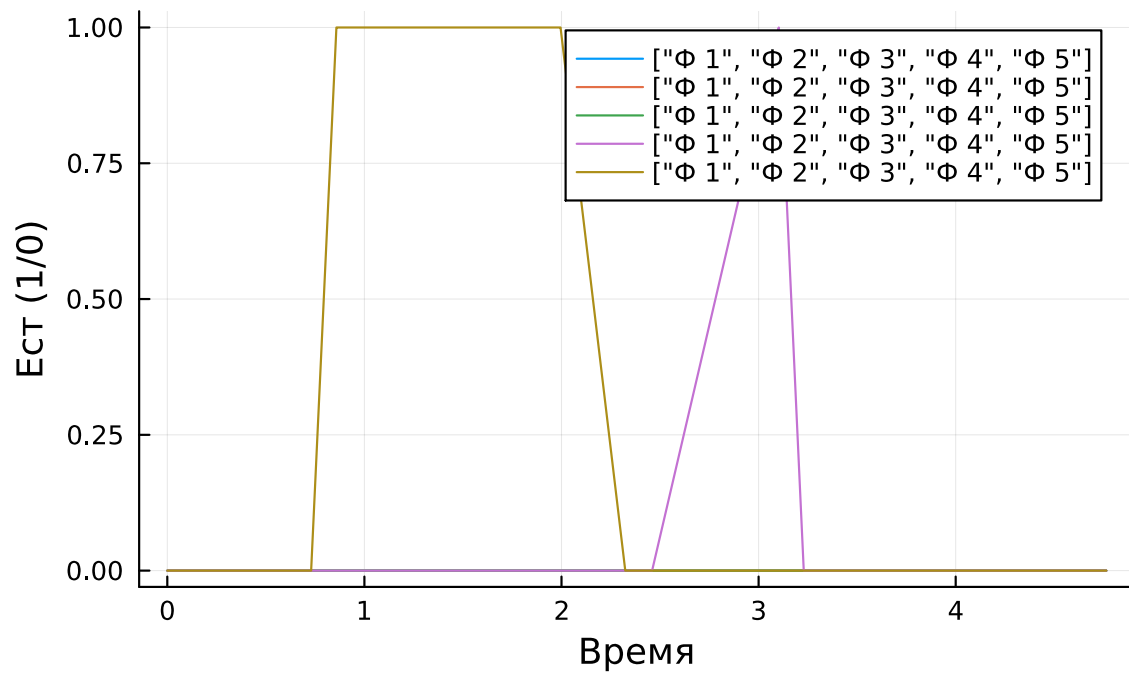
11 Классическая сеть Петри

Строим временные зависимости состояния Eat_i для каждого философа в классической сети.

По оси X откладывается модельное время, а по оси Y — значение позиции Eat_i .

```
p1 = plot(  
    df_classic.time,  
    Matrix(df_classic[:, eat_cols]),  
    label = [" $\phi_i$ " for i = 1:N],  
    xlabel = "Время",  
    ylabel = "Ест ( $1/\theta$ )",  
    title = "Классическая сеть",  
)
```

Классическая сеть



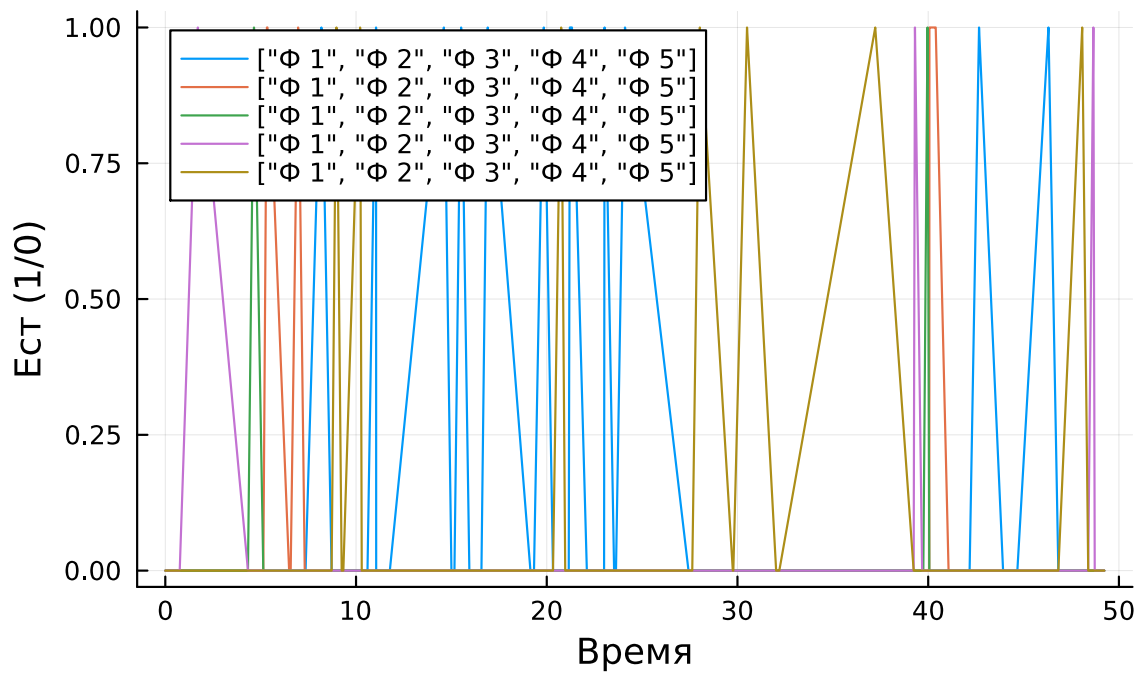
12 Сеть Петри с арбитром

Аналогичным образом строим график для сети с арбитром.

Это позволяет визуально сравнить поведение философов после введения дополнительного механизма синхронизации.

```
p2 = plot(  
    df_arbiter.time,  
    Matrix(df_arbiter[:, eat_cols]),  
    label = [" $\phi$   $i$ " for i = 1:N],  
    xlabel = "Время",  
    ylabel = "Ест ( $1/\theta$ )",  
    title = "Сеть с арбитром",  
)
```

Сеть с арбитром

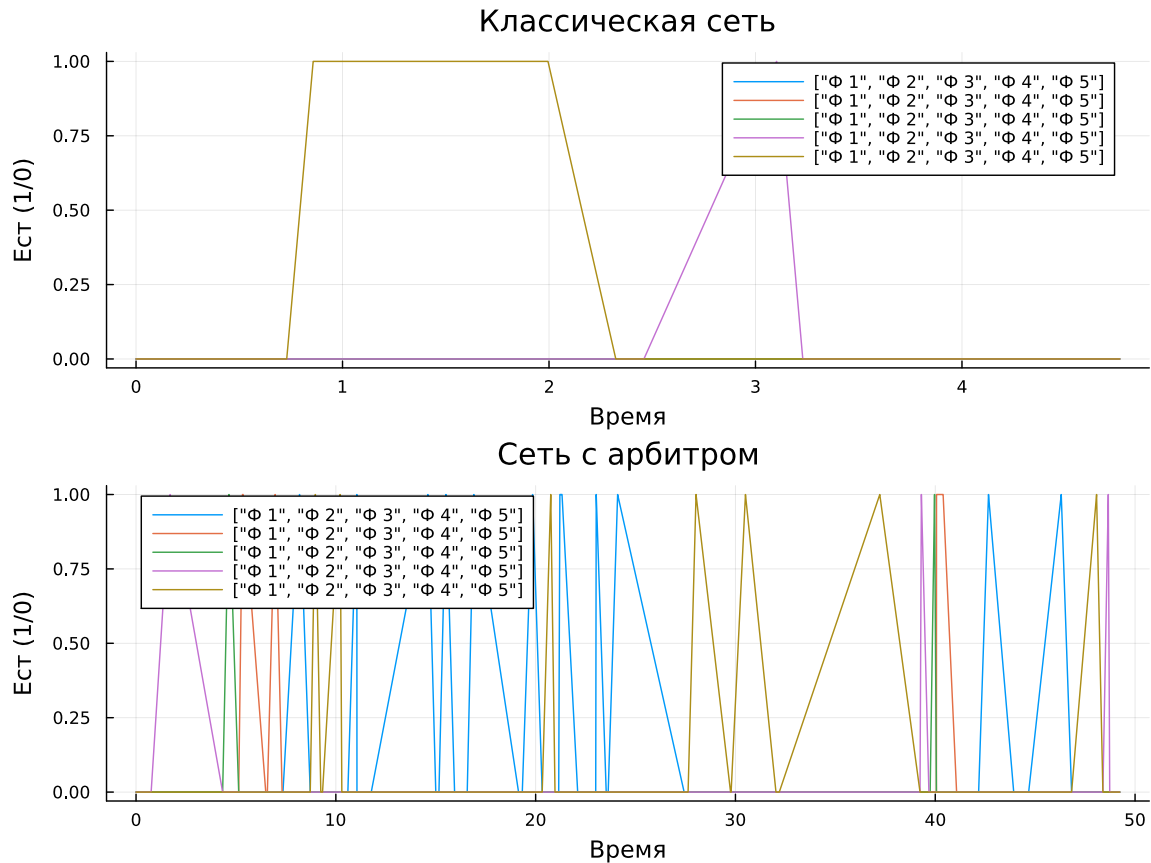


12.1 Объединение графиков

Для удобства сравнения объединяем оба графика в одну вертикальную композицию.

Верхний график соответствует классической сети, нижний — сети с арбитром.

```
p_final = plot(
    p1,
    p2,
    layout = (2, 1),
    size = (800, 600),
)
```



12.2 Сохранение итогового графика

Итоговая визуализация сохраняется в каталоге `plots` под именем `final_report.png`.

```
savefig(
    plotsdir("final_report.png"),
)
```

```
"/home/alkamal/work/study/2026-1/2026-1==study--simulation-modeling/2026-1--
study--simulation-modeling/labs/lab05/project/plots/final_report.png"
```

После завершения выводим сообщение о расположении сформированного графика.

```
println(  
    "Отчёт сохранён в plots/final_report.png",  
)
```

Отчёт сохранён в plots/final_report.png

13 Вывод

Итоговый график позволяет сравнить динамику состояния Eat_i для всех пяти философов в двух вариантах сети Петри.

Классическая сеть используется для наблюдения поведения системы без дополнительного ограничения на захват ресурсов.

Сеть с арбитром позволяет исследовать влияние дополнительного механизма синхронизации на последовательность состояний философов и сравнить её с классическим вариантом.

Полученный рисунок может быть использован в итоговом отчёте лабораторной работы как сравнительная визуализация двух моделей.

14 Выводы

В ходе лабораторной работы была реализована модель «Обедающие философы» с использованием сетей Петри. Выполнено моделирование классической сети и сети с арбитром. В классической сети была обнаружена взаимная блокировка, тогда как использование арбитра позволило избежать её. Результаты моделирования сохранены в виде таблиц, графиков и GIF-анимации. Также выполнено сравнение поведения двух вариантов сети и подготовлены литературные версии программ в форматах Quarto и Jupyter Notebook.